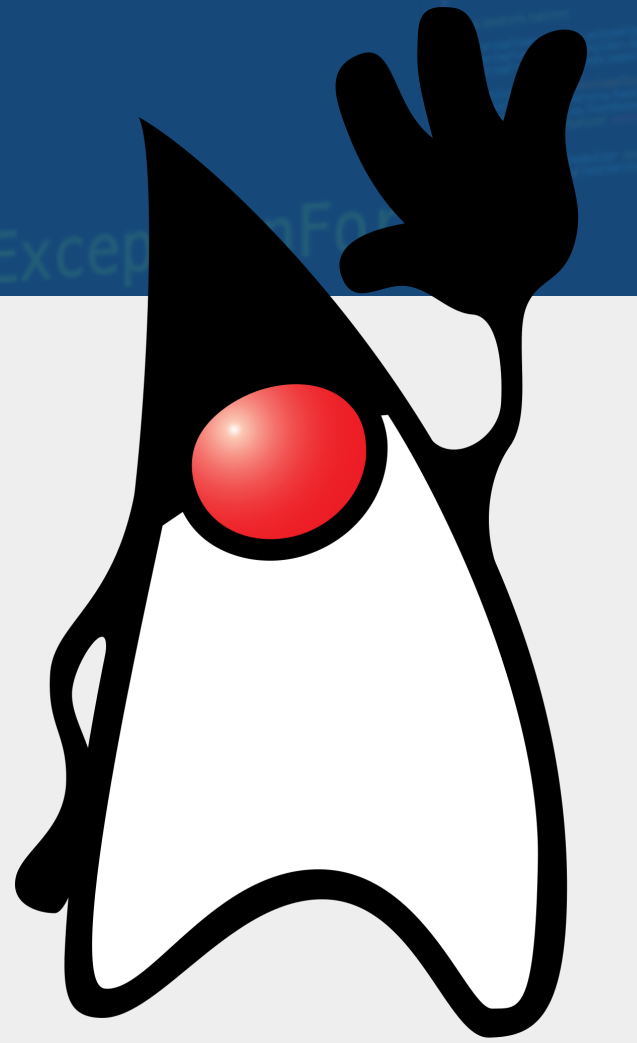


Java Programlama Dili

11. Bölüm Diziler

Akın Kaldıroğlu



Amaç



- Bu bölümde diziler (arrays) ele alınacaktır
- Daha önce ele alınan akış kontrolü yapılarına birlikte Java'nın yordamsal (procedural) kısmı büyük ölçüde tamamlanmış olacak
- Dolayısıyla yordamsal yöntemle gerçekleştirilebilecek algoritmik programlamanın temellerini ele almış olacağız
- Sonraki konular tamamen nesnelerle ilgili olacak



- **Diziler**
 - Dizi Tanıtmak
 - Elemanlara Erişim
 - for each
 - Parametre Olarak Dizi
 - Çok Boyutlu Diziler
- **Arrays Sınıfı**
- **main Metoda Parametre Geçme**
- **Algoritmalar**
- **Ödevler**
- **Bundan Sonra?**

Diziler



Torbalar (Collections)

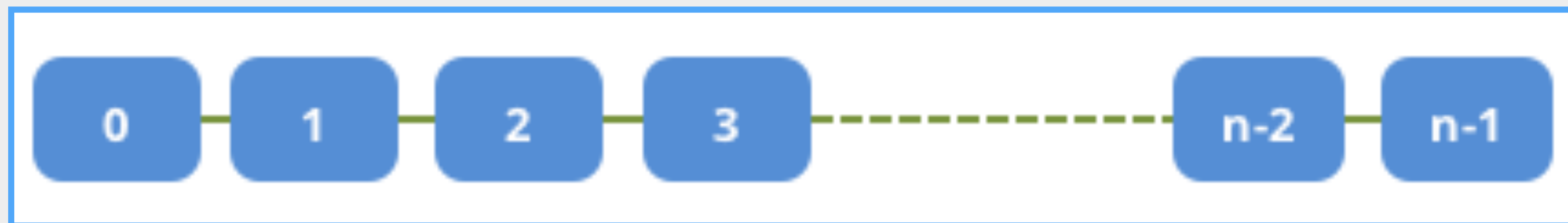


- Bütün dillerde olduğu gibi Java'da da, birden fazla ilkel değişkeni ya da nesneyi yönetmeyi sağlayan **torba (collection)** yapıları vardır
- Java'nın `java.util` paketinde zengin bir **torba çerçevesi (collection framework)** vardır,
 - Bu torbaları ileride ayrı bir bölümde ele alacağız
- Bu bölümde, bu yetenekli torbalardan önce, en temel torba yapısı olan, **diziyi (array)** ele alacağız

Diziler (Arrays) - I



- Dizi, aynı tipten, belirli sayıdaki elemanı, dizili (ordered) bir şekilde tutan bir veri yapısıdır (data structure)
- Diziler ile birden fazla değer, topluca tek bir nesne olarak yönetilebilir
- Dizilerin elemanları, **oda/hücre** (**cell**) denen yapılarda tutulur
- Değişkenlerle ilgili kutu benzetmesi odalar için de düşünülebilir



Diziler (Arrays) - II



- Dizilerin, aynı zamanda kısıt da olan iki özelliği vardır:
 - Dizilerin uzunlukları yani kapasiteleri sabittir, belirli sayıda eleman tutabilir ve bu bilgi dizi oluşturulurken verilmelidir,
 - Diziler, homojen veri yapılarıdır, elemanları aynı tipten, ilkel ya da referans, olmalıdır
- Bu iki özellikten dolayı diziler hızlıdır ama özellikle sabit bir kapasiteye sahip olmaları sıklıkla bir kısıt haline gelir
- Bu kısıtları olmayan torbalar Java'nın torba çerçevesinde vardır



Dizi Tanıtmak

Dizi Tanıtımı - I



- Dizilerin tipi vardır ve bu tip, dizinin içinde tutulacak elemanların tipidir
 - Dizinin sadece tek bir tipten elemanlara sahip olabileceğini biliyoruz
- Dizi tanıtımı, referans değişkeni tanıtımı gibidir; tipi ve referansın ismiyle yapılır çünkü diziler nesnedir
- Farklılık, diziyi göstermek üzere kullanılan dizi sembolüdür: []

```
ElementType[] arrayName;
```

```
int[] intArray;  
Pizza[] pizzas;  
Student[] students;
```

Dizi Tanıtımı - II



- [] sembolünün nerede olduğunun önemi yoktur!
- [] sembolü tipten ya da referanstan sonra olabilir
- Yaygın kullanım []'nin tipten sonra olmasıdır

```
ElementType[] arrayName;  
int[] intArray;  
  
ElementType [] arrayName;  
int [] intArray;  
  
ElementType[]arrayName;  
int[]intArray;  
  
ElementType []arrayName;  
int []intArray;  
  
ElementType arrayName[];  
int intArray[];
```

Dizi Tanımlamak - I



- Dizi tanımlamak için dizinin boyutuna ihtiyaç vardır
 - Dizinin boyutu `int` tipinde 0 ya da pozitif tamsayı bir değerdir
 - Dolayısıyla en çok $2^{31}-1$ ösall bir dizi oluşturulabilir
- Dizinin boyutu `new`'den sonra tip bilgisiyle birlikte kullanılır; kurucu (constructor) çağrısı yapılmaz, bu çağrıyı JVM yapar

```
ElementType[] arrayName = new ElementType[size];  
ElementType arrayName[] = new ElementType[size];
```

Dizi Tanımlamak - II



- Bu şekilde, belirtilen sayıda odaya sahip olan bir dizi oluşturulur
- Oluşturulan dizi nesnesine referansıyla ulaşılır

```
ElementType[] arrayName = new ElementType[size];
```

```
int[] intArray = new int[20];  
Pizza []pizzas = new Pizza[5];  
Student students[] = new Student[5_000];
```

- Dizi tanımlanırken boyut olarak negatif sayı verilemez
- Bu durumda `java.lang.NegativeArraySizeException` fırlatılır

Dizi Elemanlarının İlk Değeri - I



- Bir dizi oluşturulduğunda, odalarındaki elemanlar, dizinin tipinin varsayılan (default) değerine sahip olur
- Bu değer `boolean` için `false`, diğer yedi sayısal ilkel tip için ise 0'ın bir türüdür
- `char` sayılar için kullanılmasa da sayısal karşılıklara sahiptir
- Referans tipler için ise varsayılan değer `null`'dir

```
int[] intArray = new int[20];  
System.out.println(intArray[19]); // Prints 0  
Pizza[] pizzas = new Pizza[5];  
System.out.println(pizzas[0].name); // NullPointerException
```

Dizi Elemanlarının İlk Değeri - II



- Dolayısıyla bir dizi nesnesi oluşturmak ile o dizinin odalarına değer atamak farklı şeylerdir; dizi oluşturulunca odaları varsayılan değerde bırakılmamalı ve ilk değer atanmalıdır
- Bu amaçla dizinin elemanlarına tek tek erişerek değer ataması yapmak gerekir

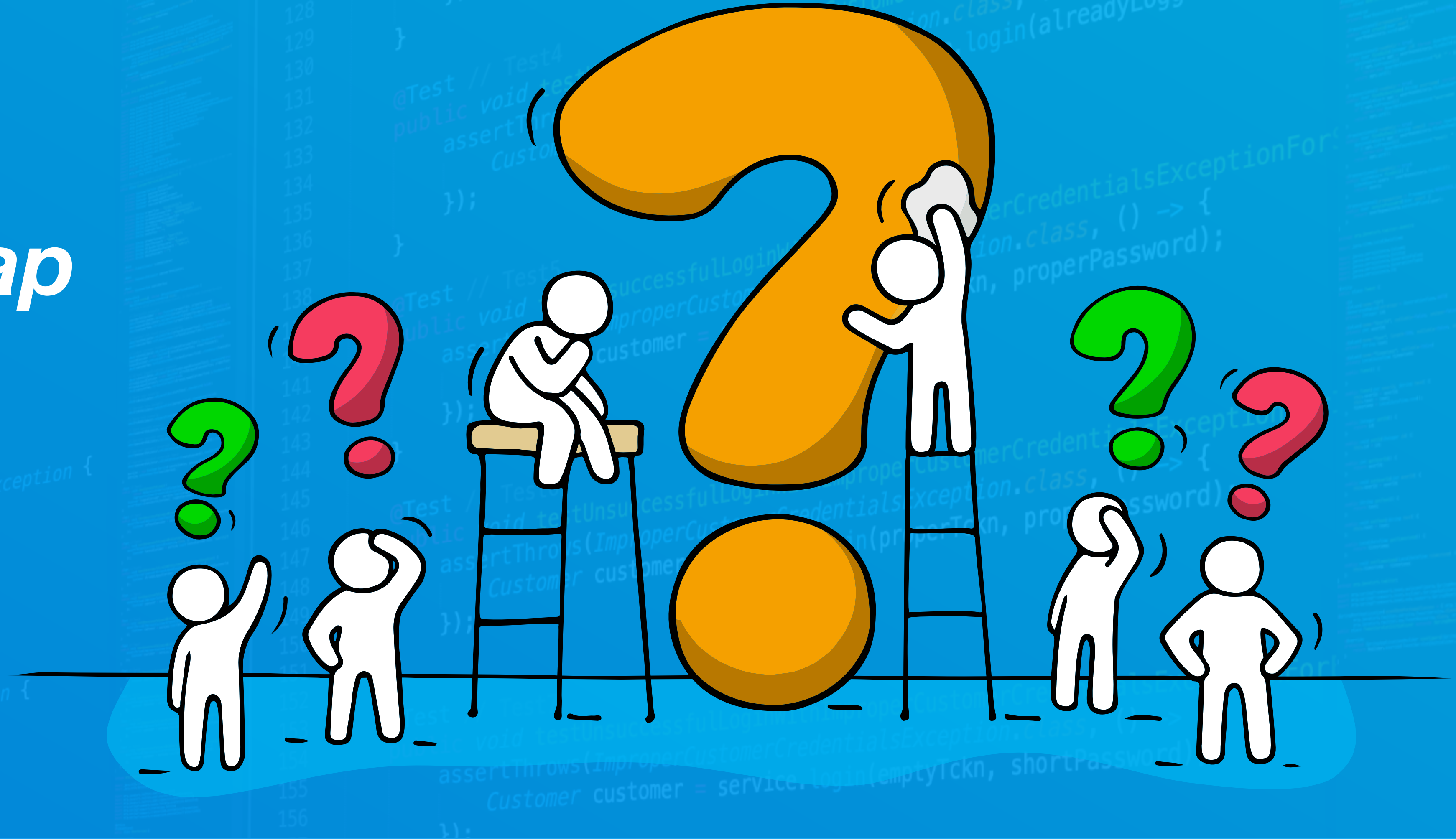
Diziler Nesnedir



- Java'da diziler nesnedir, diziyi gösteren değişken de dizi nesnesini gösteren bir referanstır
- Dizinin referansına `null` atanabilir
- Diziler üzerinde uzunluğunu yani dizinin oda sayısını veren `length` isimli `int` tipinde `final` bir alan bulunur ve bu alanın değeri dizinin referansı ile okunabilir ama atama yapılamaz

```
int[] intArray = new int[20];
System.out.println(intArray.length); // Prints 20
intArray.length = 10; // Error!
...
intArray = null;
```

Soru ve Cevap Zamanı!





Elemanlara Erişim

Dizi Elemanlarına Erişim



- Dizi elemanlarına referansıyla ve `[]` içinde dizinin oda numarası, indisi (index) ile erişilir
- Oda numarası 0'dan başlar ve dizinin uzunluğunun bir eksiğine kadar (`length-1`) devam eder
- Erişim hem okumak hem atamak için yapılabilir



```
int i = intArray[intArray.length - 1]; // i is 0
intArray[0] = 10; // Assigning 10 to the first cell
System.out.println(intArray[0]); // Prints 10

Pizza pizza = pizzas[2]; // All pizzas are null now
pizzas[0] = new Pizza(); // Assigning a new pizza
```

Oda Numaraları - I



- Dizinin elemanlarına erişirken oda numarası olarak `int` tipinde bir değişken ya da bir sabite kullanılmalıdır
- Oda erişiminde `long` kullanılamaz ama `int`'e otomatik olarak çevrilen `byte`, `short` ve `char` kullanılabilir
- Odalara erişimde 0'dan küçük veya `length-1`'den büyük bir indis kullanılması halinde `java.lang.ArrayIndexOutOfBoundsException` fırlatılır

Oda Numaraları - II



- Dolayısıyla dizinin elemanlarına erişimde muhakkak bu sınırların kontrolü yapılmalıdır

```
int[] array;  
...  
if(i ≥ 0 && i < array.length){  
    int value = array[i];  
    ...  
}
```

Dizi Elemanlarına İlk Değer Atama - I



- Dizideki odalara ilk değer atamanın ilk yolu, odalara tek tek ulaşarak atama yapmaktır
- Bu durumda `for` ya da `while` döngüsü kullanılır
- Numarasıyla odalara ulaşılırken sınır değerler olan 0 ve `length-1`'e dikkat edilmesi gereklidir

```
Random r = new Random();
for (int i = 0; i < intArray.length; i++) {
    intArray[i] = r.nextInt(100);
}
```

```
Random r = new Random();
int i = 0;
while(i ≤ intArray.length - 1) {
    intArray[i] = r.nextInt(100);
    i++;
}
```

Dizi Elemanlarına İlk Değer Atama - II



- Dizi oluşturulurken içinde saklanacak veriler biliniyorsa, bu değerler { } içinde virgül ile ayrılarak doğrudan verilebilir
- Bu şekilde ilk değer verirken **new** kullanılmaz; uzunluk da verilmez; uzunluk değerlerin sayısından çıkarılır

```
ElementType[] arrayName = { initial values };
```

```
int[] array = {1,2,3,4,5,6,7,8,9,0};  
Pizza[] pizzas = {new Pizza(), new Pizza(), null};  
boolean[] b = {true, false};
```

Dizi Elemanlarına İlk Değer Atama - III



- Bu şekilde ilk değer verme, tanıtmayla bir arada olmalıdır aksi taktirde derleme hatası olur

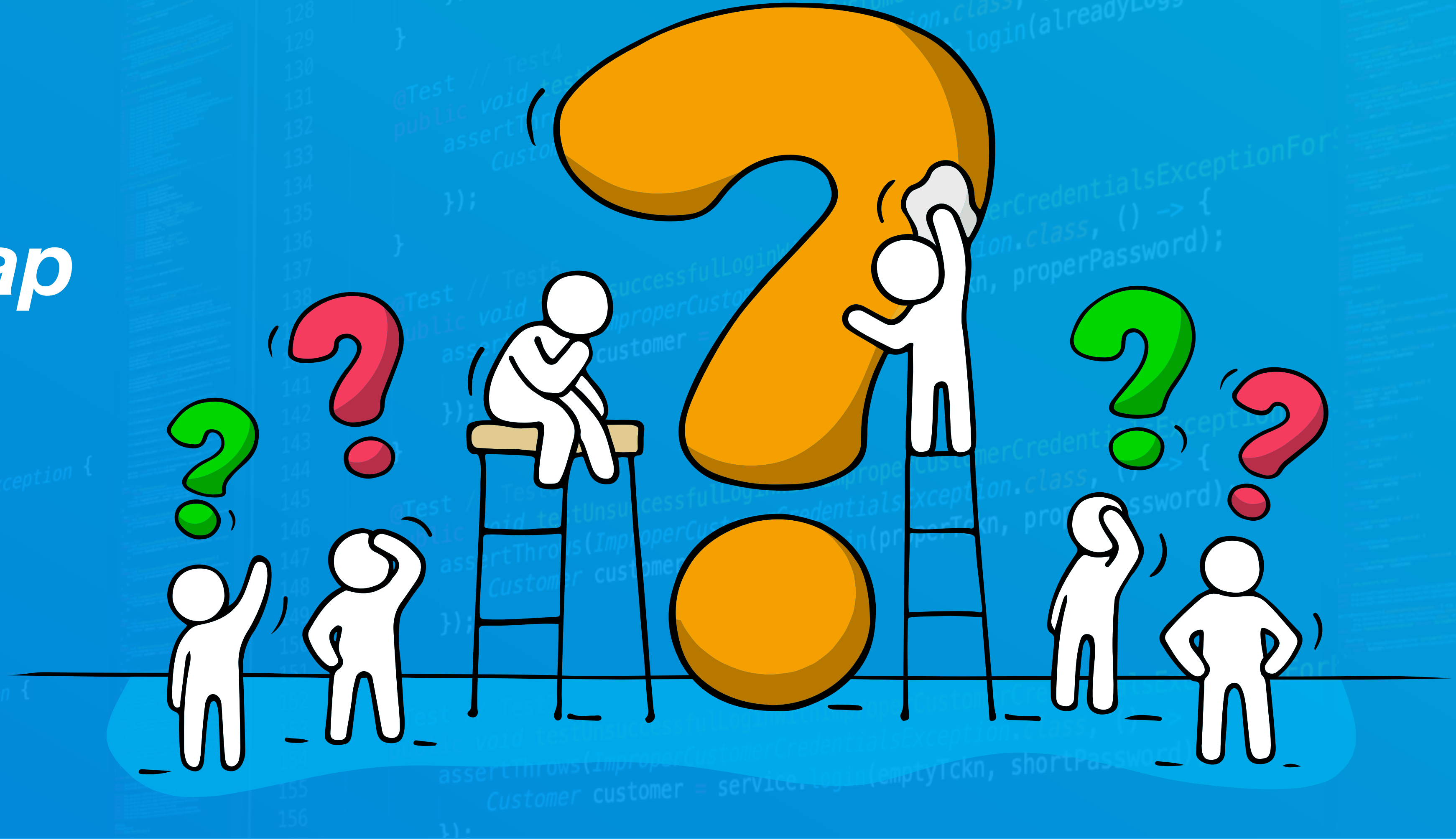
```
boolean[] b;  
b = {true, false}; // Error!
```

ArrayDemo



- `ArrayDemo`

Soru ve Cevap Zamanı!





for each

For Each - I



- `for`'un **her biri** (**for each**) anlamında bir yapısı daha vardır
- Bu, dizi gibi torbalar üzerinde çalışıp, bir indis kullanmadan, elemanlarına **tek tek ulaşmak** (**iteration**) için özel bir `for` yapısıdır
- Bu özel yapı **for each** olarak okunur ama sadece farklı bir `for` yapısıdır
- `for`'un her döngüsünde *element* ile bir sonraki değere erişilir

```
for(type element : array)
```

```
int[] array ...;  
for(int i : array)  
    System.out.println(i);
```

```
int[] array ...;  
for(int i = 0; i < array.length; i++)  
    System.out.println(array[i]);
```

For Each - II



- Bu **for each** anlamındaki **for** yapısı sadece **okuma** amaçlıdır (**read only**), derleme ya da çalışma zamanında hata vermese bile aşağıdaki kodun bir etkisi yoktur:

```
int[] intArray = new int[10];
for(int i : intArray)
    i = 5;    // No effect!
```

- Fakat dizinin tipi referans ise, bu durumda odadaki referansların durumlarında değişiklik yapılabilir

```
Pizza[] pizzas = new Pizza[3];
... // Initialization
for(Pizza p : pizzas)
    p.name = "Quattro Formaggi"; // Making all pizzas formaggi
```

ForEach



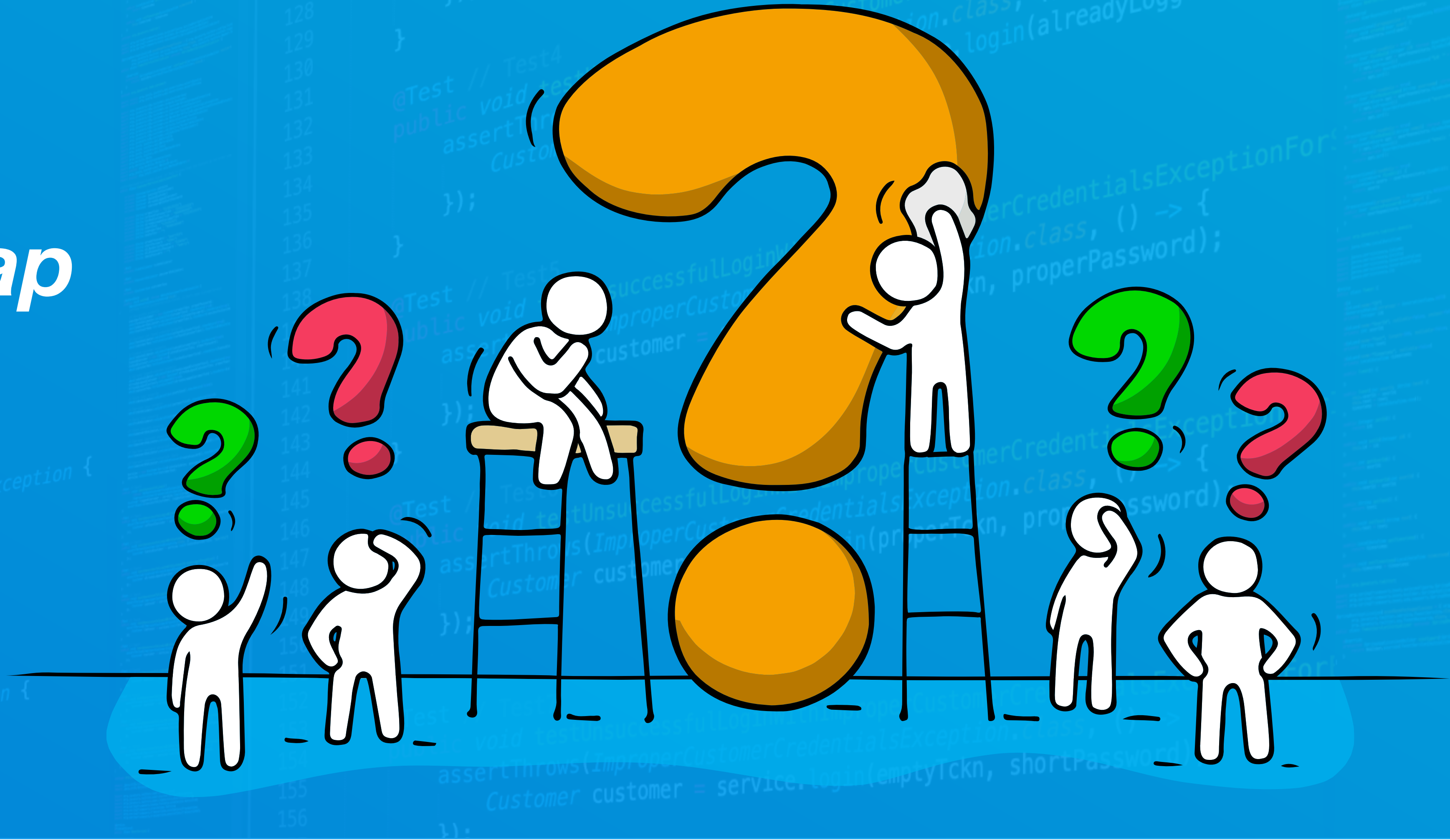
- ForEach

ReverseArray



- `ReverseArray`

Soru ve Cevap Zamanı!





Parametre Olarak Dizi

Metotlara Parametre Olarak Geçme



- Dizileri metotlara parametre olarak geçmek için bir kaç farklı şekil vardır
- Daha önce tanımlanmış bir dizinin referansı geçilebileceği gibi, nesnelere benzer şekilde, referans kullanmadan **new** ile oluşturulan dizi nesnesi de geçilebilir

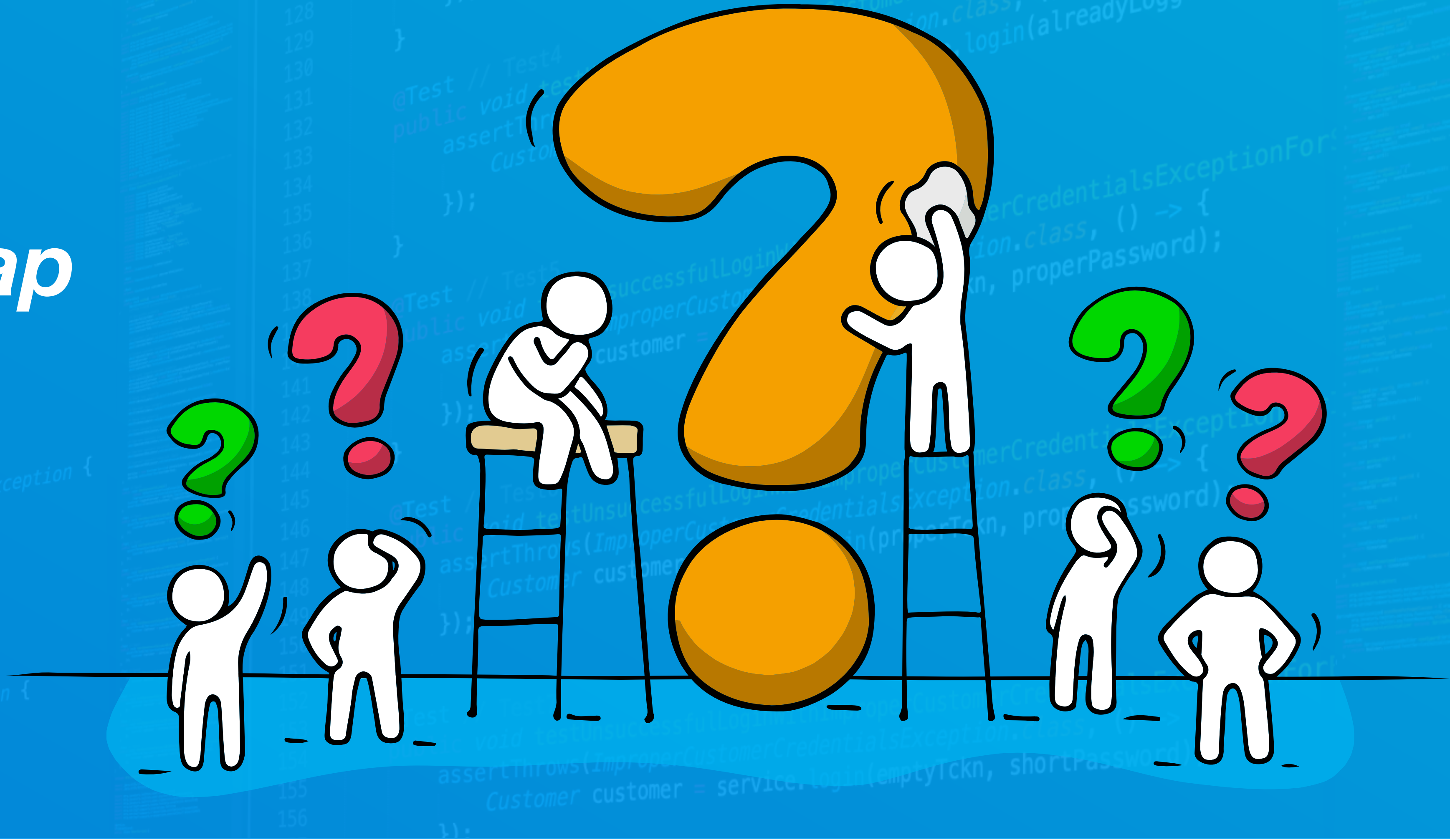
```
void calculateSum(int[] array){...}  
  
int[] array1 = new int[3];  
calculateSum(array1); //Passing the reference of the array  
  
int[] array2 = {81, 19, -14};  
calculateSum(array2); //Passing the reference of the array  
  
calculateSum(new int[]{43, 25, 99}); // OK  
calculateSum({43, 25, 99}); //Error! Type info missing!  
calculateSum(new int[3]); //OK, Passing with default values!
```

ArrayAsParameter



- `ArrayAsParameter`

Soru ve Cevap Zamanı!





Çok Boyutlu Diziler

Çok Boyutlu Diziler - I



- Çok boyutlu diziler (multi-dimensional arrays), dizi içinde dizi olarak ifade edilirler
- Bir başka deyişle çok boyutlu dizi elde etmek için, ilk dizinin her bir odasına bir başka dizi konur
- Çok boyutlu diziler, matris gibi çok boyutlu yapılar için kullanılabilir

```
ElementType[][] name = new ElementType[size][];
```

```
int[][]coordinates = new int[4][];
```

Çok Boyutlu Diziler - II



- Çok boyutlu dizi tanımlarken, boyut sayısı kadar [] kullanılmalıdır
- Ayrıca sadece esas yani ilk dizinin boyutu verilmelidir
- Tanımlama sırasında içteki dizilerin boyutlarının verilmesine ihtiyaç yoktur
- Çünkü içteki diziler farklı boyutlarda olabilir
- Örneğin iki boyutlu dizi söz konusuysa, dizi, matris olmak zorunda değildir, içteki diziler farklı boyutlarda olabilir

Çok Boyutlu Diziler - III



- Çok boyutlu diziler kısa yolla, doğrudan elemanları verilerek tanımlanabilir

```
ElementType[][] name = {{...}, {...}, .., {...}}
```

```
int[][] coordinates = {{3,7}, {13,2}, {8,5}};  
int[][] points = {{1,2,3}, {4,5,6}, {7,8,9}};  
int[][] a = {{1,2,3}, {4,5}, {6}, {}};
```

Çok Boyutlu Dizileri İşlemek - I



- Çok boyutlu dizilerde iç içe diziler vardır dendi:
- En dıştaki dizinin elemanları da ayrı birer dizidir
- Örneğin 2 boyutlu bir dizide, en dıştaki dizinin elemanları tek boyutlu bir dizidir

```
int[][] coordinates = {{3,7}, {13,2}, {8,5}};  
int[] coordinates1 = coordinates[0]; //{3,7}  
int[] coordinates2 = coordinates[1]; //{13,2}  
int[] coordinates3 = coordinates[2]; //{8,5}  
  
int i1 = coordinates[0][0]; // 3  
int i2 = coordinates[0][1]; // 7  
int i3 = coordinates[1][0]; // 13  
int i4 = coordinates[1][1]; // 2  
int i5 = coordinates[2][0]; // 8  
int i6 = coordinates[2][1]; // 5
```

Çok Boyutlu Dizileri İşlemek - II



- Çok boyutlu dizilerin işlenmesi için boyut sayısı kadar iç içe döngüye ihtiyaç vardır
- İç döngülerde, iç dizilerin boyutu alınmalıdır

Çok Boyutlu Dizileri İşlemek - III



```
void print(int[][] array){  
    for(int i = 0; i< array.length; i++)  
        for(int j = 0; j < array[i].length; j++)  
            System.out.println(array[i][j]);  
}
```

```
void print(int[][][] array){  
    for(int i = 0; i< array.length; i++)  
        for(int j = 0; j < array[i].length; j++)  
            for(int k = 0; k < array[i][j].length; k++)  
                System.out.println(array[i][j][k]);  
}
```

MultiDimArray



- `MultiDimArray`



- * kullanarak verilen bilgilerle aşağıdaki şekilleri dizi kullanarak çizen programları, **while**, **do-while** ve **for** kullanarak yazın:
- Yükseklik ile dik üçgen,
- Yükseklik ile eşkenar üçgen ve baklava dilimi (diamond)
- Yükseklik ve genişlik ile dikdörtgen
- Şekilleri içleri dolu ve boş olarak çizebilirsiniz

Ödev



```
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
```

```
*****
*                               *
*                               *
*                               *
*                               *
*                               *
*                               *
*                               *
*                               *
*                               *
*****
```

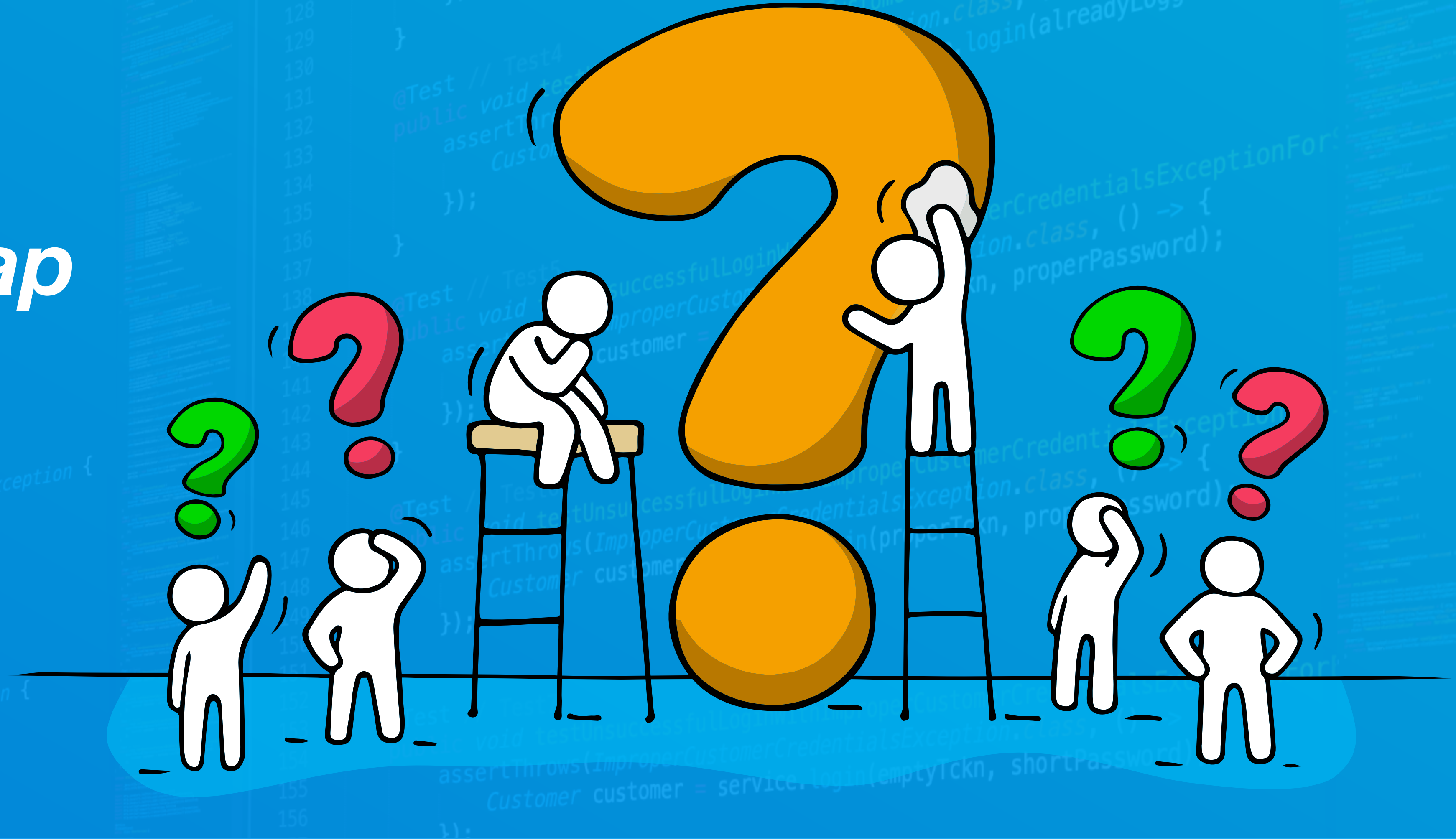
```
*
***
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
```

```
      *
     ***
    *****
   ********
  *********
 ***
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
```



- Matris çarpımı yapan bir program yazınız
- Bu amaçla önce boyut bilgisini sonra da buna uygun olarak elemanlarını dışarıdan alarak iki tane matris oluşturun
- Sonra matrisin çarpımını yapın ve sonucu basın

Soru ve Cevap Zamanı!



Arrays Sınıflı



Arrays Sınıfı



- `java.util` paketindeki **Arrays** sınıfı, dizilerle ilgili kolaylık sağlayıcı (utility) metotlara sahiptir:
 - `copyOf`
 - `copyOfRange`
 - `fill`
 - `sort`
 - `binarySearch`
- <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Arrays.html>



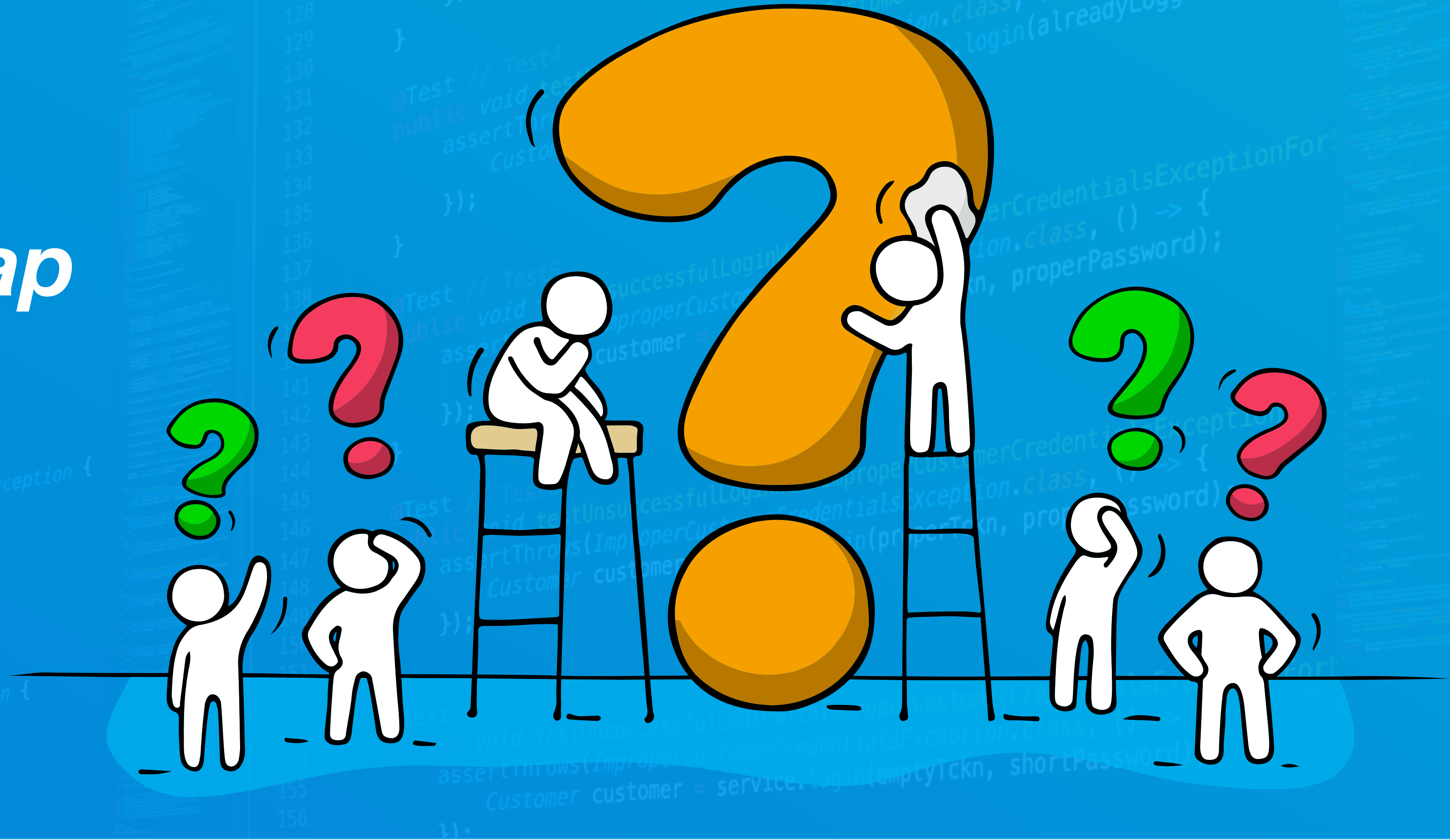
- **ArraysDemo**
- **Arrays** sınıfının API'sine bakarak, metotlarının hangi tipten argüman aldıklarına ve özelliklerine dikkat edin

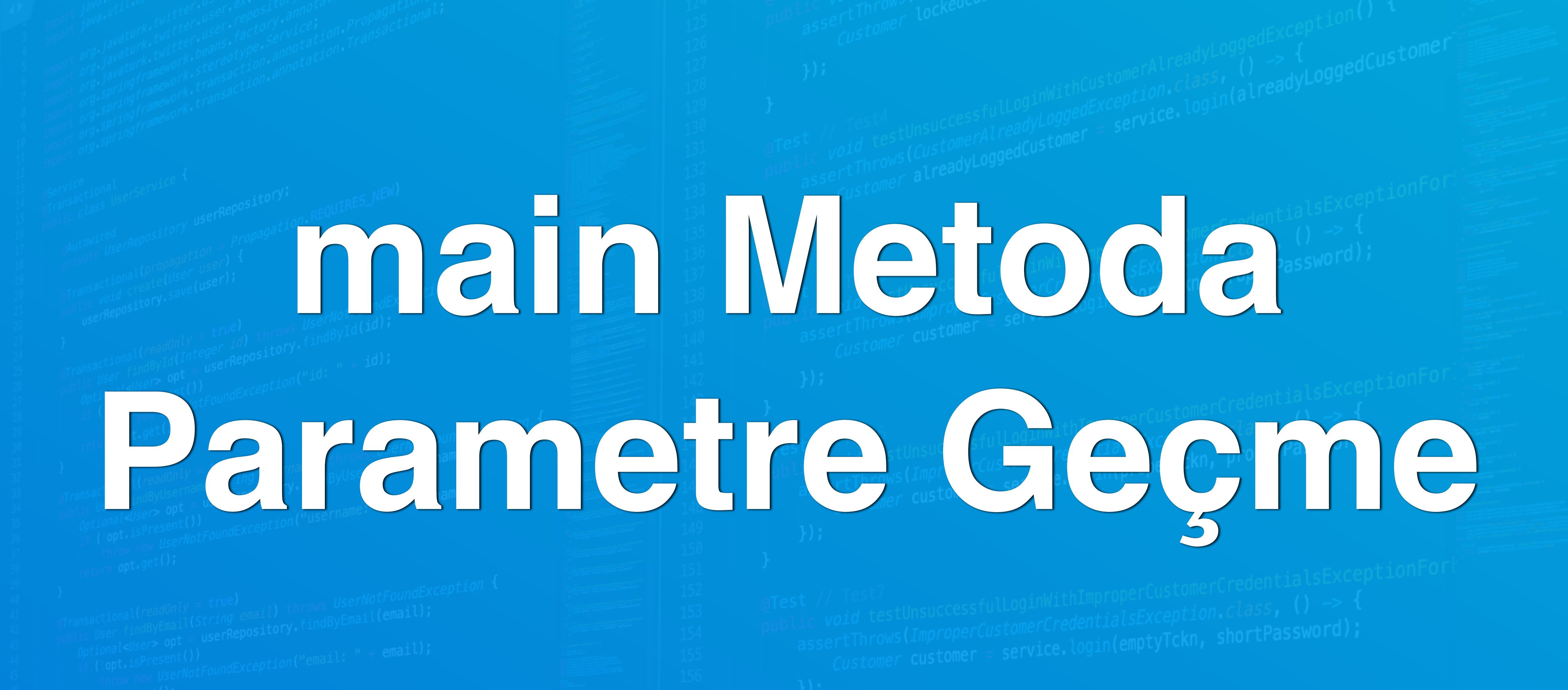
Sıralanamayan Nesneler



- İlkel tipler **tabii-doğal** olarak **sıralanabilen** (**naturally orderable**) tiplerdir
- Örneğin
 - `1 < 2`, `a < b` ya da `false < true`
- `Student` ya da `Car` gibi sınıfların nesnelerinin nasıl sıralanabilecekleri ileride ele alınacaktır

Soru ve Cevap Zamanı!





main Metoda Parametre Geçme



main Metoda Parametre Geçme - I



- `main` metoda sahip olan sınıf `java` ile çalıştırılırken `main` metoda parametre geçilebilir
- Geçilen parametreler, `main` metodun argümanı olan `String` dizisinde saklanır

```
public static void main(String[] args)
```

```
public static void main(String . . . args)
```

- `main` metotlu sınıf bu amaçla çalıştırılırken, komut satırında ya da IDE'lerde parametreler verilmelidir

main Metoda Parametre Geçme - II



- `main` metoda geçilen parametrelerin 0'dan başlayan indislerle `String` olarak alınabileceğine, gerekirse `length` ile uzunluk kontrolü yapmak gerektiğine dikkat edin

```
public static void main(String[] args){
    if(args.length == 3){
        String name = args[0];
        String lastName = args[1];
        String age = args[2];
        ...
    }
    else{
        System.out.println("Please provide three arguments!");
        System.exit(1);
    }
}
```

main Metoda Parametre Geçme - II



- Eğer aralarında boşluk olan kelimeler tek bir parametre olarak geçilecekse kelimeler " " arasına alınmalıdır

```
public static void main(String[] args){  
    if(args.length == 3){  
        String name = args[0];  
        String lastName = args[1];  
        String age = args[2];  
        ...  
    }  
    ...  
}
```

```
java ClassName Ali Saglam 32
```

```
java ClassName "Mehmet Ali" Saglam 32
```

main Metoda Parametre Geçme - IV



- Java 25 ile birlikte gelen *instance main method* özelliğiyle, `main` metodun basitleştiğini ve bu kapsamda `String` dizi parametre almasının da seçimli hale geldiğini hatırlıyoruz

```
void main()
```

- Ama bu yeni halinde de `main` metota parametre geçilebilir

```
void main(String[] args)
```

```
void main(String . . . args)
```

main Metoda Parametre Geçme - V



```
void main(String[] args){  
    if(args.length == 3){  
        String name = args[0];  
        String lastName = args[1];  
        String age = args[2];  
        ...  
    }  
    ...  
}
```

```
java ClassName Ali Saglam 32
```

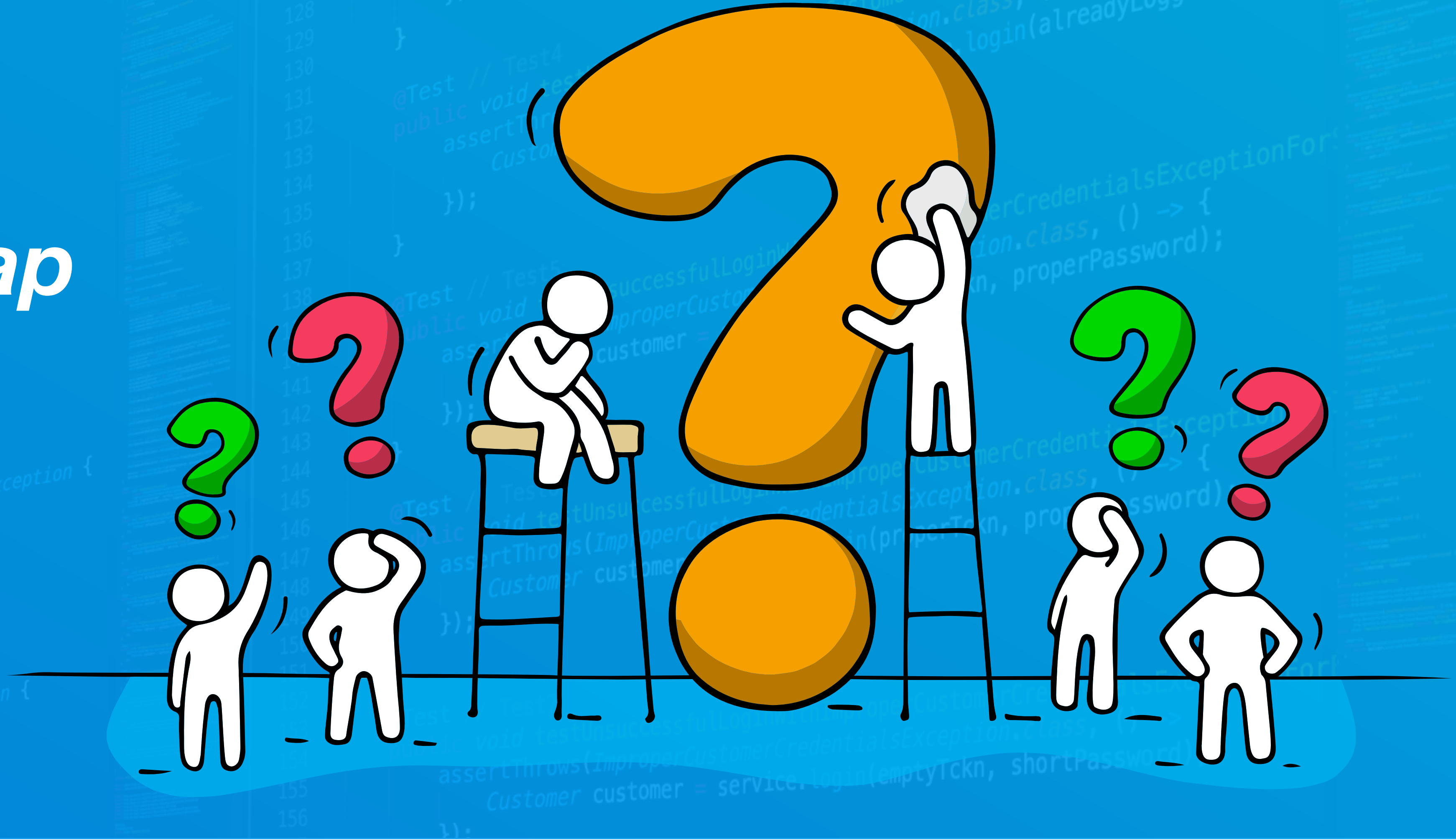
```
java ClassName "Mehmet Ali" Saglam 32
```

MainDemo



- `MainDemo`
- Bu örneği hem komut satırından hem de IDE'nizden çalıştırın.
- IDE'lerde genelde *Run* ayarlarında `main` metoda parametre geçme alanı olur

Soru ve Cevap Zamanı!



Algoritmalar





- Merriam-Webster'da algorithm terimi şöyle tanımlanmaktadır:

algorithm: a procedure for solving a mathematical problem (as of finding the greatest common divisor) in a finite number of steps that frequently involves repetition of an operation

broadly : a step-by-step procedure for solving a problem or accomplishing some end

- <https://www.merriam-webster.com/dictionary/algorithm>

Algoritmalar



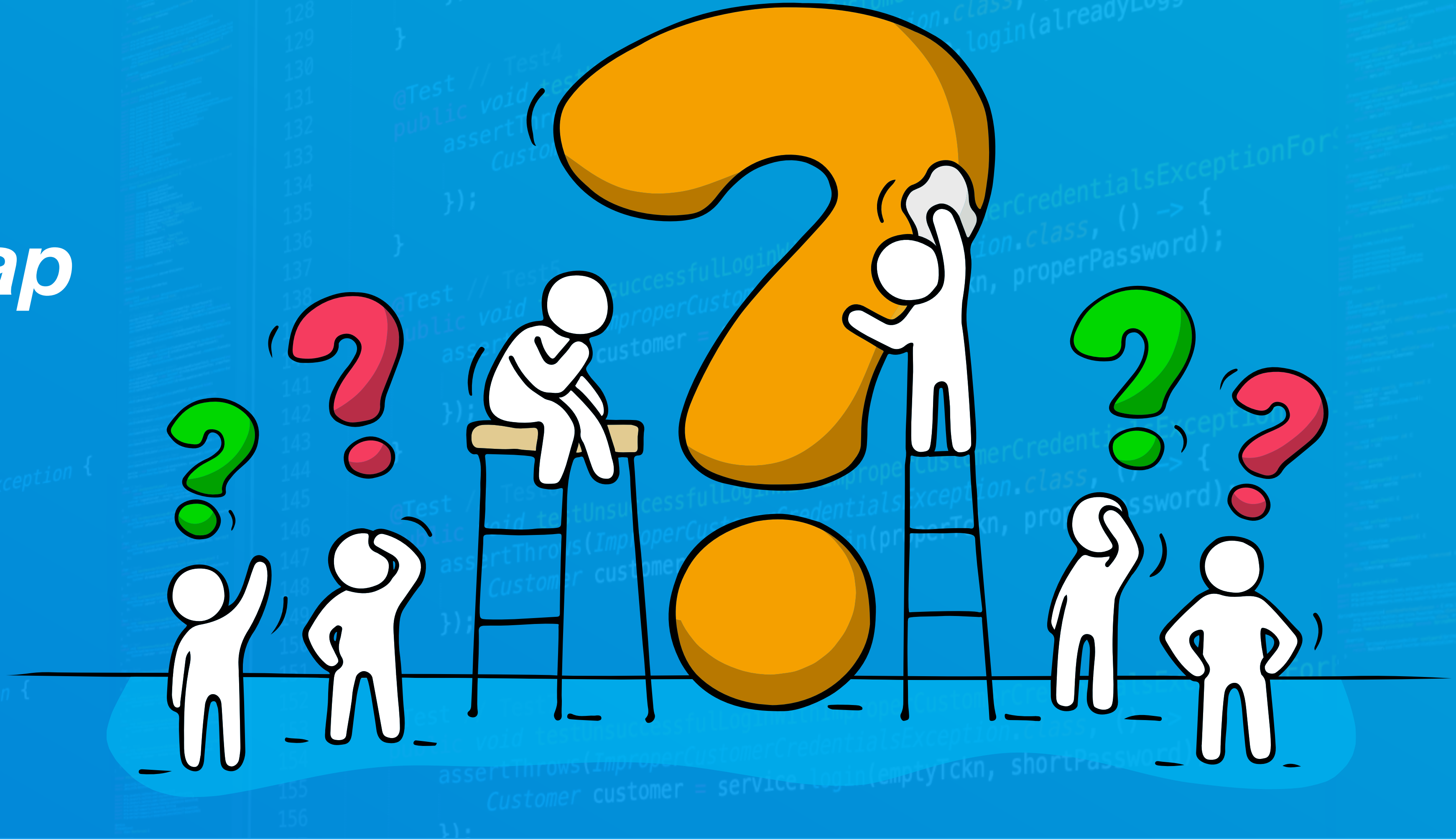
- Diziler pek çok algoritmada yer alır çünkü birden fazla değer olduğu torbalar sıralama (sorting) ya da arama (searching) işlemlere konu olur
- Matrisler, fonksiyonlar, çizgeler (graphs) hep birden fazla değer tarafından temsil edilen matematiksel yapılardır ve algoritmalara konu olurlar
- Sıralama için de pek çok algoritma vardır
- Bubble Sort da bunlardan birisidir

BubbleSort



- BubbleSort

Soru ve Cevap Zamanı!



Ödevler





1. Girilen bir dizideki elemanları, sıralarını bozmadan ama tekil olarak yeni bir diziye aktaran bir program yazın. Örneğin girdi {3, 3, 87, 56, 1, 87, 3, 2 } ise çıktı {3, 87, 56, 1, 2 } olmalıdır.
2. Girilen bir sayıya kadar olan asal sayıları **Sieve of Eratosthenes**'in algoritmasını kullanarak bulunuz. Algoritma hakkında bilgi için http://en.wikipedia.org/wiki/Sieve_of_Eratosthenes adresini kullanabilirsiniz.



3. Bir tam sayının bölenlerini bulan bir programı dizileri kullanarak yazınız.
4. Bir tam sayının asal bölenlerini bulan bir programı dizileri kullanarak yazınız.

Fundamental theorem of arithmetic (the unique factorization theorem or the unique-prime-factorization theorem): Every integer greater than 1 either is prime itself or is the product of prime numbers, and that this product is unique, up to the order of the factors.

https://en.wikipedia.org/wiki/Fundamental_theorem_of_arithmetic

$$100 = 2^2 * 5^2, 1000 = 2^3 * 5^3, 17248 = 2^5 * 7^2 * 11 \text{ 68}$$



5. `main` metoda geçilen ve içinde boşluk barındıran bir `String`'i tersine çevirin; yani “I Love Java” girilince çıktı “avaJ evoL I” olsun
- a. Bunun için `String` sınıfındaki `charAt(int index)` metoduna indis geçerek nesnenin o indisindeki karakteri alınabilir
 - b. `String` nesnesinde kaç karakter olduğunu öğrenmek için `length()` metodu kullanılır



- c. `String` nesnesindeki karakterleri tek tek almak için 0'dan `String` nesnesinin uzunluğunun bir eksiğine kadar gidilerek `charAt(int index)` metodu kullanılabilir
- d. `char[]`'den `String` oluşturmak için de `String`'in şu kurucusunu (constructor) kullanabilirsiniz: `String(char[] array)`

Bundan Sonra?



Bundan Sonra?



- Bir sonraki bölümle nesne merkezli programlamaya giriş yapacağız.

Bölüm Sonu

Soru ve Cevap Zamanı!

